


# RISK CLASSIFICATION

Engineering implementation note / 01 — Decision boundaries

I am Amin Azimi, AI Architect and developer of Aegis Agent. I organize action governance around explicit risk labeling, consented proposals and human authorization. I distinguish policy signals from enforcement and distinguish local execution from external side effects.

## 01 | Four-level action taxonomy

I classify action titles with deterministic, case-insensitive substring rules: L0 observation, L1 planning or drafting, L2 state-changing operations and L3 higher-impact external or irreversible operations. Higher-risk keywords take precedence. The classification is a heuristic, not semantic analysis or a proof of safety.

## 02 | Effect-aware safeguards

I use an explicit allow-list of known local effects for L0 when an effect or action kind is supplied. Unknown effects receive at least L1. Certain allow-listed kinds, including proposal kinds, may remain L0 even if the title includes planning terms. Because the allow-list includes a local .eml outbox write, I do not equate the L0 label with literal absence of filesystem writes.

## 03 | A separate authority score

I implement a second risk evaluator in HumanAuthority. It combines task length, selected keywords and tenant sensitivity into a bounded score. Its labels are AUTO, SINGLE_APPROVAL, DUAL_APPROVAL and DENY. These are distinct from the L0–L3 action labels. A DUAL_APPROVAL classification is not, by itself, evidence that two independent approvals are collected.

## 04 | Proposal creation

I generate deterministic twin action proposals from a previously committed, consented profile. The baseline kinds include digest review, repository review when repositories exist, and weekly planning. I store proposed actions in SQLite; generating a proposal does not itself authorize execution.




# APPROVAL & EXECUTION

Engineering implementation note / 02 — State transitions

### 05 | Digest-bound approval

I calculate a SHA-256 digest over a canonical action envelope containing action identity, tenant, kind, title, payload, effect, risk level and policy version. Approval checks that the action is still proposed, that the tenant matches and that the supplied digest matches the current envelope. The approval decision, actor and timestamp are persisted.

### 06 | Explicit rejection and feedback

I support rejection with structured reason labels and an optional explanation. The action and its decision feedback are recorded in SQLite. Where a correlation ID exists, the implementation attempts to associate audit events with the same action. Some auxiliary logging and receipt calls use best-effort exception handling; I do not promise that every auxiliary artifact is always written.

### 07 | Local execution boundary

I separate the action state machine — proposed → approved / rejected → executed — from actual external capability. The twin-action execution slice is local and records state or produces local work products and receipts. The documented email path writes a local .eml outbox rather than sending live SMTP mail. I do not claim autonomous payments, deployments or unrestricted external execution.

### 08 | Threat model and limitations

I do not treat title-based keyword classification as a comprehensive authorization system. Attackers can choose misleading wording, and the risk engine does not infer every side effect from code behavior. Effective protection also depends on trusted identity binding, route-level checks, action-specific permissions, and verification of the actual execution path.

### 09 | Source-backed engineering position

I base these statements on core/twin_risk.py, core/human_authority.py and core/twin_actions.py. Source inspection establishes the described branches and state transitions, not an independently executed penetration test or an external compliance assessment.

ACTIVE DEVELOPMENT — I continue to strengthen risk semantics, authorization coverage, identity binding and execution-path verification before making production-wide enforcement claims.